# APPLIED MACHINE LEARNING AND DATA ENGINEERING AT PREPROD CORP

## A INTERNSHIP REPORT

*Submitted By*

**SWAPNENDU BANIK**

**22BHI10005**

*A report submitted in fulfillment of the requirements for the Award of Degree of*

**BACHELOR OF TECHNOLOGY**

In

**COMPUTER SCIENCE AND ENGINEERING**

**(HEALTH INFORMATICS)**

Under Supervision of

**Dr. Pratosh Kumar Pal**

*Assistant Professor*

*School of Computer Science Engineering and Artificial Intelligence (SCAI)*

**(Duration: 1st September, 2024 to 15th December, 2024)**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**VIT BHOPAL UNIVERSITY**

March 2026

**VIT BHOPAL UNIVERSITY, KOTHRIKALAN, SEHORE**

**MADHYA PRADESH – 466114**

# BONAFIDE CERTIFICATE

This is to certify that the internship report entitled **"Applied Machine Learning and Data Engineering at PreProd Corp"** is the bonafide work of **SWAPNENDU BANIK (22BHI10005)** who carried out the internship at **PreProd Corp** from **1st September, 2024** to **15th December, 2024**, under the guidance of **Vansh** (Technical Mentor, PreProd Corp), **Krishnav Sir** (CEO, PreProd Corp) and **Harish Sir** (CTO, PreProd Corp).

**PROGRAM CHAIR,**

**Dr. T Venkateswara Rao**
School of Computing Science Engineering
and Artificial Intelligence (SCAI),
VIT Bhopal University.

**PROJECT SUPERVISOR,**

**Dr. Pratosh Kumar Pal**
Assistant Professor
School of Computing Science Engineering
and Artificial Intelligence (SCAI),
VIT Bhopal University.

The Semester Internship – DSN2093 Examination is held on

_____

# INTERNSHIP CERTIFICATE

preprodcorp

CERTIFICATE

OF INTERNSHIP — 15 DEC 2024

PROUDLY PRESENTED TO

Swapnendu Banik

In recognition of your valuable contribution and commitment towards a group internship starting from 1 September 2024 to 15 Dec 2024, we proudly present this certificate of completion of internship, acknowledging your involvement as a Data Analyst for the group.

KRISHNAV **DAVE**
Founder & CEO
PreProd Corp

HARISH **S**
Co-founder & CTO
PreProd Corp

TPPC_Cpnsrc_CID_35

***[Link to Certificate]***

# ACKNOWLEDGEMENT

First and foremost, I would like to extend my sincere gratitude to my academic supervisor, **Dr. Pratosh Kumar Pal**, Assistant Professor, School of Computing Science Engineering and Artificial Intelligence (SCAI), VIT Bhopal University, for his invaluable guidance, encouragement, and institutional support throughout the preparation of this internship report. His mentorship ensured that my work remained aligned with the academic requirements and professional standards expected of this programme.

I wish to express my deepest professional gratitude to the PreProd Team at **PreProd Corp** for welcoming me into their team and providing a learning environment of exceptional quality. In particular, I am profoundly thankful to **Vansh**, my primary technical mentor, whose patience and expertise guided me through the most demanding phases of the internship — from setting up Kafka on WSL to building end-to-end ML pipelines. I am equally grateful to **Harish Sir**, **Krishnav Sir**, and **Mohini**, whose domain expertise, insightful feedback, and constant encouragement shaped the quality of my work across every module. Krishnav Sir's mentorship during the Amazon Review Analysis individual project was particularly formative.

I also wish to express my gratitude to the faculty of the **School of Computing Science Engineering and Artificial Intelligence (SCAI)**, VIT Bhopal University, for their continuous academic support and for fostering an environment that encourages real-world learning experiences such as this internship.

Last but not least, I am deeply indebted to my parents and friends, whose unconditional encouragement, belief in my abilities, and unwavering support have been the greatest motivation throughout this journey.

**SWAPNENDU BANIK**

**22BHI10005**

# LIST OF FIGURES


| Figure No. | Title | Page No. |
|---|---|---|
| Figure 1.1 | Agile-Scrum Sprint Cycle Diagram | 6 |
| Figure 1.2 | Apache Kafka Architecture – Producers, Brokers and Consumers | 7 |
| Figure 1.3 | DVC Workflow – Local Workspace, Cache and Remote Storage | 9 |
| Figure 1.4 | DQN Agent-Environment Interaction Loop | 10 |
| Figure 1.5 | YOLO Dog Breed Detection – Sample Inference Output | 12 |

# LIST OF TABLES


| Table No. | Title | Page No. |
|---|---|---|
| Table 1.1 | Comparison of Agile vs. Waterfall Methodology | 6 |
| Table 1.2 | Kafka CLI Commands Reference | 7 |
| Table 1.3 | Comparison of Spark vs. Hadoop | 8 |
| Table 1.4 | Tree-Based Models – Classifier and Regressor Notebooks | 8 |
| Table 1.5 | MLFlow Pipeline Stages – Scripts, Inputs and Outputs | 9 |
| Table 1.6 | DQN Hyperparameters and Descriptions | 10 |
| Table 1.7 | Hugging Face Models Used – Task and Model Name | 11 |
| Table 1.8 | Text Representation Techniques – Comparison Summary | 11 |
| Table 2.1 | Week 1 – Agile-Scrum Weekly Activity Log | 6 |
| Table 2.2 | Week 2 – Data Streaming Weekly Activity Log | 7 |
| Table 2.3 | Week 3 – PySpark & Tree Models Weekly Activity Log | 8 |
| Table 2.4 | Week 4 – MLOps Weekly Activity Log | 9 |
| Table 2.5 | Week 5 – Reinforcement Learning Weekly Activity Log | 10 |
| Table 2.6 | Week 6 – NLP Weekly Activity Log | 11 |
| Table 2.7 | Week 7 – YOLO & Amazon Reviews Weekly Activity Log | 12 |

# TABLE OF CONTENTS


| Chapter No. | Title | Page No. |
|---|---|---|
| | List of Figures | i |
| | List of Tables | ii |
| 1 | **CHAPTER 1: LEARNING OBJECTIVES** | 1-4 |
| | 1.1 Introduction | 1 |
| | 1.2 Motivation for the Work | 1-2 |
| | 1.3 Background | 2 |
| | 1.4 Problem Statement | 2 |
| | 1.5 Objectives of the Work | 3 |
| | 1.6 Summary of the Internship | 4 |
| 2 | **CHAPTER 2: WEEKLY OVERVIEW OF INTERNSHIP ACTIVITIES** | 5-12 |
| | 2.1 Introduction | 5 |
| | 2.2 Major Activities | 5 |
| | 2.3 Learning Objectives / Internship Objectives | 5 |
| | 2.4 Summary | 5 |
| | 2.5 Weekly Briefings | 6-12 |
| 3 | **CHAPTER 3: INTRODUCTION** | 13-14 |
| 4 | **CHAPTER 4: INTERNSHIP DISCUSSION** | 15-21 |
| | 4.1 Agile Framework & Team Dynamics | 15 |
| | 4.2 Data Streaming with Apache Kafka | 16 |
| | 4.3 PySpark & Tree-Based Machine Learning Models | 17 |
| | 4.4 MLOps Pipeline – MLFlow, DVC & DagsHub | 18 |
| | 4.5 Reinforcement Learning & Deep Q-Network | 19 |
| | 4.6 NLP with Hugging Face Transformers | 20 |
| | 4.7 Object Detection with YOLO & Amazon Review Analysis | 21 |
| 5 | CHAPTER 5: CONCLUSION | 22 |
| | Note on Internship Schedule | 23 |
| | Bibliography | 24 |

# CHAPTER 1: LEARNING OBJECTIVES

## 1.1 Introduction

This chapter outlines the foundational framework of the internship undertaken at PreProd Corp from September to December 2024. Internships serve as a critical bridge between the theoretical knowledge acquired in an academic setting and the practical, real-world application demanded by the technology industry. For a student of Computer Science and Engineering, this transition from classroom to workplace is not merely desirable — it is essential for developing the adaptive, problem-solving mindset that defines a competent professional.

The internship at PreProd Corp's PreProd Team was structured around this philosophy. Rather than assigning a single, narrowly defined project, the team designed a progressive, multi-domain learning trajectory spanning seven productive weeks. Each week was dedicated to a distinct technical domain — ranging from Agile methodology and data streaming to reinforcement learning and transformer-based NLP — ensuring broad exposure while maintaining sufficient depth in each area.

## 1.2 Motivation for the Work

The motivation for pursuing this internship was rooted in a recognition of the gap between academic ML education and industry practice. While university coursework provides rigorous grounding in algorithms, mathematics, and programming, it rarely exposes students to the full complexity of production ML systems — the infrastructure, the tooling, the team dynamics, and the iterative, often non-linear nature of real-world model development.

Modern machine learning is not just about training accurate models. It is about building reliable data pipelines, tracking experiments systematically, versioning data and models, deploying in production environments, and maintaining systems over time. The internship at PreProd Corp was motivated by the desire to experience and contribute to precisely these dimensions of ML engineering — to understand not just how to build a model, but how to build it well, within a functioning team, using industry-standard tools.

Additionally, the breadth of the internship — spanning data engineering (Kafka, PySpark), MLOps (MLFlow, DVC), advanced ML (reinforcement learning, transformer fine-tuning), and computer vision (YOLO) — was particularly appealing because it

mirrored the multi-disciplinary nature of real data science and ML engineering roles, where practitioners must operate comfortably across the full stack.

## 1.3 Background

PreProd Corp is a technology-focused organisation whose PreProd Team operates as an agile, sprint-driven unit at the intersection of machine learning, data engineering, and software development. The team follows the Agile-Scrum framework with structured sprint cycles, daily stand-ups, sprint reviews, and retrospectives. This operational structure ensures that work is continuously prioritised, progress is transparent, and learnings are captured systematically.

The technical landscape within which the internship was situated is characterised by rapid evolution. Apache Kafka has become the de facto standard for real-time data streaming in enterprise systems. PySpark extends the reach of data processing to scales beyond what a single machine can handle. MLFlow and DVC address the reproducibility and versioning challenges endemic to ML research and development. Transformer-based models, pioneered by BERT and GPT, have fundamentally transformed the capabilities and expectations of natural language processing. YOLO represents the state of the art in real-time object detection. Understanding and working with this ecosystem is not optional for a modern ML practitioner — it is foundational.

## 1.4 Problem Statement

The overarching challenge addressed by this internship can be stated as follows: how does an undergraduate student, equipped with theoretical ML knowledge but limited industry experience, develop genuine practical competence across the full stack of modern machine learning engineering within a constrained timeframe?

This challenge decomposed into several domain-specific sub-problems across the seven weeks. How does one configure and operate a distributed event streaming system (Kafka) on a local development machine? How does one build, tune, and evaluate ensemble ML models at scale using PySpark? How does one systematically track, version, and reproduce ML experiments using MLOps tooling? How does one design a reward function and implement experience replay to train a Deep Q-Network agent? How does one leverage pre-trained transformer models for NLP tasks and adapt them to domain-specific datasets through fine-tuning? Each of these questions represented a concrete problem that demanded investigation, implementation, and reflection.

## 1.5 Objectives of the Work

The following objectives guided the seven-week internship engagement:

- To understand and apply the Agile-Scrum framework within an active team, including sprint planning, daily stand-ups, sprint reviews, and retrospectives.
- To gain hands-on experience with Apache Kafka for real-time data streaming, including setup on WSL, producer-consumer implementation using Quixstreams, and understanding of Kafka's architectural components.
- To explore PySpark as a distributed data processing and ML framework, and to apply it for data manipulation and model-building workflows.
- To build, evaluate, and tune tree-based machine learning models — Decision Trees, Random Forests, Gradient Boosting, and XGBoost — across varied datasets for both classification and regression tasks.
- To implement a complete MLOps pipeline integrating MLFlow for experiment tracking, DVC for data versioning, and DagsHub as a collaborative remote platform.
- To understand reinforcement learning theory and implement a Deep Q-Network (DQN) agent capable of solving the CartPole sequential decision-making problem.
- To explore the Hugging Face Transformers ecosystem for NLP tasks including sentiment analysis, text generation, question answering, and summarisation, and to build a real-world review sentiment analysis pipeline.
- To train a YOLO object detection model on a domain-specific dog breed dataset and perform inference on novel images.
- To independently complete the Amazon Review Analysis project, comparing text representation techniques (BoW, TF-IDF, Word2Vec) and fine-tuning DistilBERT for sentiment classification.
- To develop professional skills including technical documentation, collaborative problem-solving, and iterative project delivery under mentorship.

## 1.6 Summary of the Internship

The internship at PreProd Corp unfolded across seven structured weeks, each anchored to a distinct technical domain. Week one established the Agile-Scrum methodology that would govern all subsequent work. Week two immersed the intern in the world of distributed data streaming through Apache Kafka and Quixstreams. Week three broadened the focus to distributed computing with PySpark and an extensive survey of tree-based machine learning algorithms. Week four addressed the MLOps lifecycle through a spike ticket integrating MLFlow, DVC, and DagsHub. Week five presented the most intellectually demanding challenge: implementing a Deep Q-Network for reinforcement learning. Week six explored the transformative capabilities of Hugging Face Transformers for NLP, culminating in a real-world Yelp review sentiment analyser. Week seven combined YOLO-based object detection for dog breed identification with an independent NLP project on Amazon Review Analysis, involving both classical text representation techniques and transformer fine-tuning.

Throughout the internship, the Agile framework provided structure and discipline, mentors provided technical depth and professional guidance, and the sprint-based delivery model ensured that every week concluded with tangible, documented deliverables. The breadth of domains covered, the depth of practical implementation required, and the quality of mentorship received made this internship a genuinely transformative professional experience.

# CHAPTER 2: WEEKLY OVERVIEW OF INTERNSHIP ACTIVITIES

## 2.1 Introduction

This chapter provides a structured account of the activities undertaken across each week of the internship. The PreProd Team's Agile-Scrum framework meant that work was naturally organised into weekly sprint-like cycles, each with defined objectives, daily activities, and deliverables. The weekly overviews presented here reflect the actual progression of learning and output, organised thematically by technical domain.

## 2.2 Major Activities

The internship encompassed seven major activity areas, each corresponding to one working week: Agile-Scrum methodology and team onboarding; Apache Kafka and real-time data streaming; PySpark and tree-based machine learning; MLOps tooling (MLFlow, DVC, DagsHub); Deep Q-Network reinforcement learning; NLP with Hugging Face Transformers; and YOLO object detection combined with Amazon Review Analysis. Each activity area involved a combination of conceptual learning, hands-on implementation, and documentation.

## 2.3 Learning Objectives / Internship Objectives

The weekly learning objectives were progressively structured to build on each preceding week. The Agile foundation of week one informed the sprint discipline of all subsequent weeks. The Linux and WSL skills developed during the Kafka week (week two) were drawn upon during PySpark setup in week three. The MLOps skills of week four were directly relevant to the experiment management discipline applied in weeks five, six, and seven. The NLP intuitions developed in week six provided conceptual grounding for the DistilBERT fine-tuning work of week seven. This interconnected architecture of learning objectives was not accidental — it reflected thoughtful curriculum design by the PreProd mentorship team.

## 2.4 Summary

Each week's activities are detailed in the tables below, organised by day and topic. The tables represent active working weeks only; periods of scheduled institutional activity and personal leave are addressed separately in the Note on Internship Schedule at the end of this report.

## 2.5 Weekly Briefings

**Week 1 – Agile-Scrum Framework (Table 2.1)**

| Day | Session | Activity / Topic Covered |
|---|---|---|
| Monday | Day 1 | Team orientation; introduction to PreProd Team structure, intern expectations, and internship schedule |
| Tuesday | Day 2 | Introduction to Agile philosophy; comparison of Agile, Waterfall, and Kanban; reading Agile Manifesto |
| Wednesday | Day 3 | Deep-dive into Scrum: roles (Product Owner, Scrum Master, Dev Team), events (Sprint, Stand-up, Review, Retro), artifacts (Backlog, Increment) |
| Thursday | Day 4 | Sprint planning exercise; understanding story points, backlog grooming, velocity, and the definition of done |
| Friday | Day 5 | Hands-on Scrum simulation with team; sprint review and retrospective participation; documentation of learnings |

**Table 1.1 – Agile vs. Waterfall Methodology**

| Agile | Waterfall |
|---|---|
| Iterative and incremental delivery | Sequential, phase-by-phase delivery |
| Change is welcomed at any stage | Change is costly and discouraged |
| Continuous customer collaboration | Customer involved mainly at start and end |
| Frequent, smaller deliverables per sprint | Single large deliverable at project end |

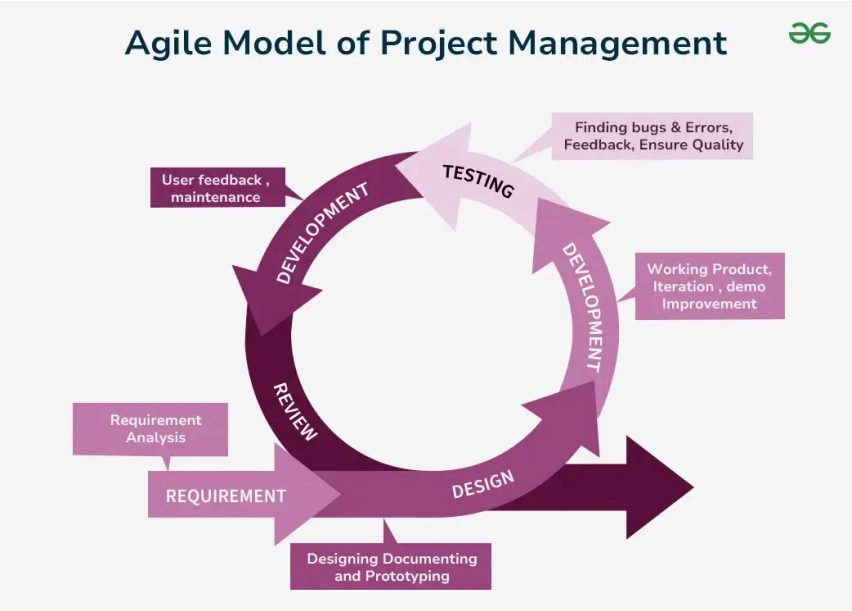


*Figure 1.1: The Agile-Scrum Sprint Cycle showing Sprint Planning, Daily Stand-up, Sprint Review, and Retrospective*

*Source Code Repository: [Link to Week-1 Codebase]*

**Week 2 – Data Streaming with Apache Kafka & Quixstreams (Table 2.2)**

| Day | Session | Activity / Topic Covered |
|---|---|---|
| Monday | Day 1 | Introduction to event-driven architecture; overview of Apache Kafka — topics, partitions, brokers, producers, consumers, offsets |
| Tuesday | Day 2 | WSL2 setup on Windows; configuring Windows Firewall (port 9092); netsh port forwarding rules to route traffic to WSL IP |
| Wednesday | Day 3 | Downloading and installing Apache Kafka in WSL; running ZooKeeper and Kafka broker; creating and deleting topics via Kafka CLI |
| Thursday | Day 4 | Setting up Python virtual environment; writing producer.py using Quixstreams and Faker; writing consumer.py; testing live data streaming |
| Friday | Day 5 | Debugging network/tunneling issues; verifying live stream in terminal; documenting Kafka CLI commands and architecture notes |

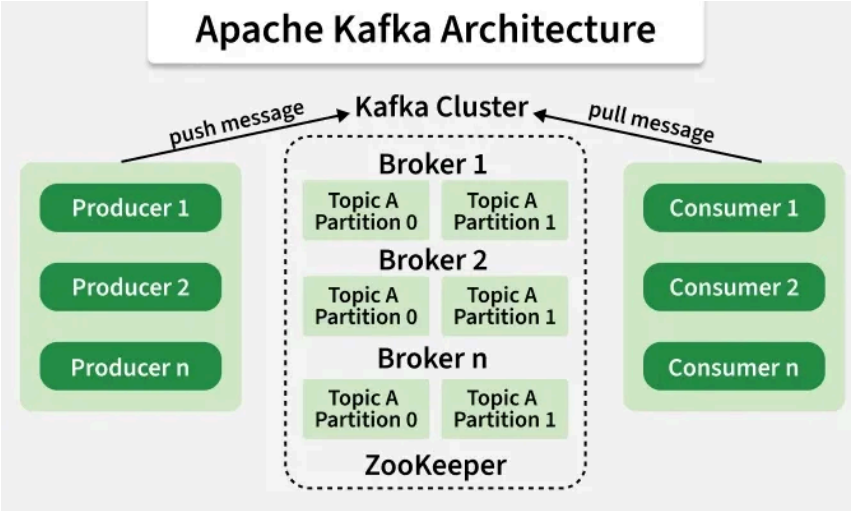


*Figure 1.2: Apache Kafka Architecture — Producers publish to Topics across Partitions; Consumers subscribe via Consumer Groups*

**Table 1.2 – Kafka CLI Commands Reference**

| Operation | CLI Command |
|---|---|
| Start ZooKeeper | *./bin/zookeeper-server-start.sh config/zookeeper.properties* |
| Start Kafka Broker | *./bin/kafka-server-start.sh config/server.properties* |
| Create Topic | *./bin/kafka-topics.sh --bootstrap-server localhost:9092 --create --topic test --partitions 1* |
| Start Producer | *./bin/kafka-console-producer.sh --topic test --bootstrap-server localhost:9092* |
| Start Consumer | *./bin/kafka-console-consumer.sh --topic test --bootstrap-server localhost:9092 --from-beginning* |

*Source Code Repository:* ***[Link to Week-2 Codebase]***

**Week 3 – PySpark & Tree-Based Machine Learning Models (Table 2.3)**

| Day | Session | Activity / Topic Covered |
|---|---|---|
| Monday | Day 1 | Introduction to Apache Spark; Spark vs. Hadoop comparison; in-memory computing model; PySpark setup and SparkSession creation |
| Tuesday | Day 2 | PySpark DataFrames and pyspark.sql module; distributed data manipulation; SQL queries via SparkSession; pyspark.ml Pipeline concept |
| Wednesday | Day 3 | Decision Tree Classifier and Regressor — training, evaluation, visualising splits; working with multiple datasets |
| Thursday | Day 4 | Random Forest (Classifier & Regressor) and Gradient Boosting (Classifier & Regressor); hyperparameter tuning with cross-validation |
| Friday | Day 5 | XGBoost Classifier and Regressor; comparative evaluation of all models; documentation of model results and observations |

**Table 1.3 – Spark vs. Hadoop Comparison**

| Apache Spark | Apache Hadoop (MapReduce) |
|---|---|
| In-memory computation | Disk-based I/O between phases |
| Up to 100x faster for iterative workloads | Slower due to frequent disk reads/writes |
| Supports batch and real-time streaming | Primarily batch processing |
| DAG-based execution engine | MapReduce paradigm (Map + Reduce steps) |

**Table 1.4 – Tree-Based Models Implemented (Classifier and Regressor Notebooks)**

| Algorithm | Classifier | Regressor |
|---|---|---|
| Decision Tree | *decision_tree_classifier.ipynb* | *decision_tree_regressor.ipynb* |
| Random Forest | *random_forest_classifier.ipynb* | *random_forest_regressor.ipynb* |
| Gradient Boosting | *gradientBoosting_classifier.ipynb* | *gradientBoosting_regressor.ipynb* |
| XGBoost | *xgBoosting_classifier.ipynb* | *xgBoosting_regressor.ipynb* |

*Source Code Repository:* ***[Link to Week-3 Codebase]***

**Week 4 – MLOps: MLFlow, DVC & DagsHub (Table 2.4)**

| Day | Session | Activity / Topic Covered |
|---|---|---|
| Monday | Day 1 | Introduction to MLOps; understanding the ML lifecycle; the need for experiment tracking and data versioning in production ML |
| Tuesday | Day 2 | MLFlow deep-dive: Tracking, Projects, Models, and Model Registry; hands-on experiment logging (parameters, metrics, artifacts) |
| Wednesday | Day 3 | DVC (Data Version Control): dvc init, dvc add, dvc push, dvc pull, dvc checkout; integration with Git for metadata tracking |
| Thursday | Day 4 | Building end-to-end pipeline: preprocessing stage (preprocess.py), training stage with GridSearchCV (train.py), evaluation stage (evaluate.py) |
| Friday | Day 5 | DagsHub integration as remote storage; pushing experiments and data versions; spike ticket review with mentor; documentation |

*Figure 1.3: DVC Workflow — model.pkl tracked via .dvc metadata; data pushed to remote storage while Git tracks the lightweight pointer*

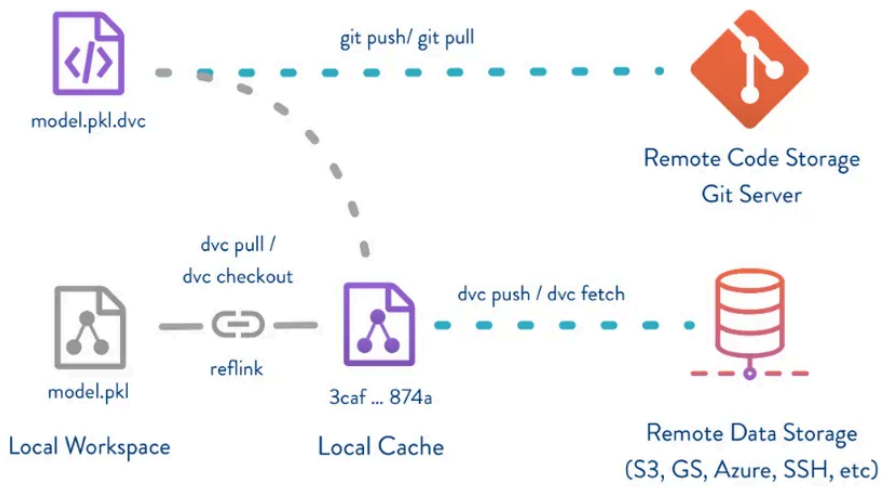


**Table 1.5 – MLFlow Pipeline Stages**

| Stage | Script | Input | Output |
|---|---|---|---|
| Preprocessing | *src/preprocess.py* | *data/raw/data.csv* | *data/processed/data.csv* |
| Training | *src/train.py* | *data/processed/data.csv* | *models/model.pkl* |
| Evaluation | *src/evaluate.py* | *models/model.pkl* | Accuracy metric logged to MLFlow |

*Source Code Repository: [Link to Week-4 Codebase]*

**Week 5 – Reinforcement Learning & Deep Q-Network (Table 2.5)**

| Day | Session | Activity / Topic Covered |
|---|---|---|
| Monday | Day 1 | Reinforcement Learning fundamentals: agent, environment, state, action, reward, policy; Markov Decision Process (MDP) formalism |
| Tuesday | Day 2 | Q-learning and the Bellman equation; exploration vs. exploitation; epsilon-greedy strategy; tabular Q-learning limitations |
| Wednesday | Day 3 | Deep Q-Networks (DQN): neural network as Q-function approximator; experience replay buffer; target network concept |
| Thursday | Day 4 | Implementing DQNAgent class: network architecture, replay memory (deque), act() and replay() methods; CartPole-v1 environment setup |
| Friday | Day 5 | Training DQN on CartPole; hyperparameter tuning (gamma, epsilon decay, learning rate, batch size); evaluating agent performance over episodes |

*Figure 1.4: DQN Agent-Environment Interaction — the agent observes state s, selects action a via DNN policy, and receives reward r from the environment*

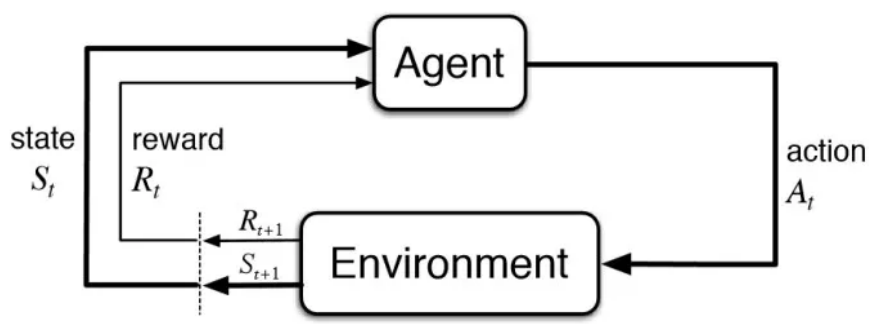


**Table 1.6 – DQN Hyperparameters and Descriptions**

| Hyperparameter | Role | Typical Value |
|---|---|---|
| Gamma (γ) | Discount factor for future rewards | 0.95 – 0.99 |
| Epsilon (ε) | Exploration rate (decays over time) | 1.0 → 0.01 |
| Epsilon Decay | Rate at which ε decreases per episode | 0.995 |
| Learning Rate | Neural network weight update step size | 0.001 |
| Batch Size | Experiences sampled from replay buffer per update | 32 – 64 |

*Source Code Repository: [**Link to Week-5 Codebase**]*

**Week 6 – NLP with Hugging Face Transformers (Table 2.6)**

| Day | Session | Activity / Topic Covered |
|---|---|---|
| Monday | Day 1 | Introduction to NLP and transformer architecture; overview of Hugging Face ecosystem, pipeline API, and tokenisation |
| Tuesday | Day 2 | Sentiment analysis using DistilBERT (SST-2); text generation with GPT-2 (Shakespeare-style prompting); exploring tokeniser outputs |
| Wednesday | Day 3 | Question answering with RoBERTa (deepset/roberta-base-squad2); text summarisation with T5 (cnicu/t5-small-booksum) |
| Thursday | Day 4 | Real-world application: scraping Yelp reviews with requests and BeautifulSoup; sentiment scoring using multilingual BERT |
| Friday | Day 5 | Aggregating sentiment scores to predict outlet rating; comparing predicted vs. actual Yelp rating; writing project documentation |

**Table 1.7 – Hugging Face Models Used**

| Task | Model | Model ID |
|---|---|---|
| Sentiment Analysis | DistilBERT | *distilbert-base-uncased-finetuned-sst-2-english* |
| Text Generation | GPT-2 | *openai-community/gpt2* |
| Question Answering | RoBERTa | *deepset/roberta-base-squad2* |
| Summarisation | T5-small | *cnicu/t5-small-booksum* |
| Review Sentiment | Multilingual BERT | *nlptown/bert-base-multilingual-uncased-sentiment* |

**Table 1.8 – Text Representation Techniques Comparison**

| Technique | Representation | Captures Context | Computational Cost |
|---|---|---|---|
| Bag of Words | Sparse term-frequency vector | No | Very Low |
| TF-IDF | Weighted sparse vector | No | Low |
| Word2Vec (Avg) | Dense mean embedding | Partial (semantic) | Medium |
| DistilBERT (FT) | Contextual deep representation | Yes (full context) | High |

*Source Code Repository:* ***[Link to NLP Codebase]***

**Week 7 – YOLO Object Detection & Amazon Review Analysis (Table 2.7)**

| Day | Session | Activity / Topic Covered |
|---|---|---|
| Monday | Day 1 | Introduction to YOLO framework; object detection vs. classification; preparing dog breed dataset in YOLO format (images/, labels/, dataset.yaml) |
| Tuesday | Day 2 | Training YOLO11n model on dog breed dataset (250 epochs, GPU on Colab); running inference on test images; reviewing detection outputs |
| Wednesday | Day 3 | Amazon Review Analysis (individual): dataset exploration; Bag of Words and TF-IDF text representation with classical ML classifiers |
| Thursday | Day 4 | Word2Vec and Averaged Word2Vec embeddings; training classifiers on dense representations; comparing model performance metrics |
| Friday | Day 5 | Fine-tuning DistilBERT on Amazon Reviews; evaluation and cross-method comparison; final report writing and submission preparation |

*Figure 1.7: Sample output from the trained YOLO dog breed detection model showing predicted bounding boxes with class labels and confidence scores*

*Source Code Repository:* ***[Link to YOLO Codebase]***

# CHAPTER 3: INTRODUCTION

The modern landscape of technology is defined not merely by isolated expertise but by the ability to navigate a diverse and interconnected ecosystem of tools, frameworks, and methodologies. For a student of Computer Science and Engineering, academic learning provides the theoretical scaffolding — but it is real-world practice that transforms knowledge into capability. This internship at PreProd Corp, spanning the period from September to December 2024, was precisely such an experience.

PreProd Corp's PreProd Team operates as an agile, sprint-driven unit working at the cutting edge of machine learning, data engineering, and AI application development. Being welcomed into this team as an intern meant immersing oneself into the actual rhythms of professional software and ML development — from morning stand-ups and sprint planning sessions to late-evening debugging marathons and the quiet satisfaction of a working pipeline.

The internship was structured across seven productive weeks, each dedicated to a distinct technological domain. This structure was not arbitrary — it reflected a thoughtfully designed learning trajectory that moved from foundational engineering practices (Agile methodology) through progressively complex technical terrain: distributed data streaming, large-scale data processing, MLOps, reinforcement learning, natural language processing, and computer vision. Each week built upon the last, creating a cumulative depth of understanding that would have been impossible to achieve through coursework alone.

Week one grounded the experience in the Agile-Scrum framework — the operational philosophy that would govern how all subsequent work was organised, executed, and reviewed. Understanding Scrum not just theoretically but by living it within an active team gave this foundational week an immediacy that textbook descriptions cannot replicate.

The second week introduced Apache Kafka and the world of real-time data streaming — an area that has become increasingly critical as organisations move toward event-driven architectures and real-time analytics. Setting up Kafka on a local machine via Windows Subsystem for Linux (WSL), navigating the labyrinthine configuration of port forwarding and network rules, and finally witnessing live data flow between a producer and consumer was both technically demanding and deeply satisfying.

Week three brought a dual focus on PySpark and tree-based machine learning models.

Apache Spark's promise of distributed, in-memory computation was explored through PySpark's Python API, while an extensive survey of tree-based algorithms — from decision trees through random forests and gradient boosting to XGBoost — provided hands-on experience with some of the most practically powerful tools in the ML practitioner's toolkit.

The fourth week addressed one of the most critical yet often overlooked aspects of production ML systems: the MLOps lifecycle. Working on a spike ticket that integrated MLFlow for experiment tracking, DVC for data versioning, and DagsHub as a collaborative remote platform gave a concrete understanding of how mature ML teams manage the complexity of iterative model development.

Perhaps the most intellectually demanding week was the fifth, dedicated to Reinforcement Learning and the implementation of a Deep Q-Network. Building an agent from scratch that could learn to balance a pole on a cart — through nothing more than interaction, reward, and the Bellman equation — was a profound demonstration of the power and patience that machine learning demands.

Weeks six and seven brought the internship into the domains of Natural Language Processing and Computer Vision. Exploring the Hugging Face Transformers library, building a Yelp review sentiment analyser, training a YOLO dog breed detector, and independently completing the Amazon Review Analysis project consolidated the broad technical exposure into a coherent professional experience.

This report documents that journey — not as a chronological log of completed tasks, but as a reflective account of what was learned, what was challenged, and what was understood more deeply as a result.

# CHAPTER 4: INTERNSHIP DISCUSSION

## 4.1 Agile Framework & Team Dynamics

The first week of the internship served as an essential foundation — not in a peripheral or perfunctory sense, but as a genuine prerequisite for everything that followed. The Agile-Scrum methodology, which the PreProd Team follows rigorously, was introduced through a combination of conceptual explanation and immediate practical application.

Agile is, at its heart, a philosophy of iterative development and adaptive planning. Where traditional waterfall models demand complete upfront specification before a single line of code is written, Agile embraces change as a feature rather than a defect. The Scrum framework operationalises this philosophy through a clear set of roles, events, and artifacts. The Product Owner holds responsibility for the product backlog and prioritisation; the Scrum Master facilitates the process and removes impediments; the Development Team owns the work within a sprint.

What made this week particularly valuable was experiencing these principles within the context of a functioning team. Sprint planning — the act of selecting and committing to a set of backlog items for the coming sprint — demonstrated how technical work is translated into discrete, estimable units. Story points, which represent relative effort rather than absolute time, were a concept that initially felt abstract but became intuitive once applied to actual tasks.

The daily stand-up, limited to three questions (what did I do yesterday, what will I do today, are there any blockers?), is deceptively simple but profoundly effective as a coordination mechanism. The sprint retrospective — perhaps the most humanly valuable Scrum event — is dedicated entirely to process improvement. Asking 'what went well, what could be better, and what will we commit to improving' at the close of each sprint creates a structured mechanism for a team to continuously evolve. Participating in these retrospectives, even as an intern, was a lesson in professional maturity and collective ownership.

The Agile foundation established in this first week permeated the entire internship. Every subsequent module was framed as a sprint, with a defined objective, daily progress, and a deliverable at the end. This structure was not bureaucratic overhead — it was the scaffolding that made the breadth of the internship manageable and coherent.

## 4.2 Data Streaming with Apache Kafka

The second week introduced a technology that, until that point, had existed only as a name in documentation — Apache Kafka. The learning curve was steep, but the depth of understanding achieved by the end of the week made it one of the most memorable weeks of the entire internship.

Apache Kafka is a distributed event streaming platform designed to handle high-throughput, fault-tolerant, real-time data feeds. Its architecture is elegantly layered: producers publish messages to named topics; brokers (server nodes) store and manage those messages across partitions; consumers subscribe to topics and process messages in real time. Partitions are the key to Kafka's scalability — by distributing a topic's data across multiple partitions, Kafka allows parallel consumption and horizontal scaling.

The setup process itself was an education. Installing Kafka on Windows Subsystem for Linux (WSL2) required configuring the Windows firewall to allow TCP traffic on port 9092, setting up port forwarding rules using netsh to route traffic from the Windows host to the WSL virtual machine, and carefully managing the WSL IP address — which changes on restart. This was the first real encounter with the messy reality of distributed systems: the code is often straightforward, but the infrastructure is not.

Once the environment was running, the focus shifted to the Kafka data pipeline itself. Quixstreams, a Python library built on top of the Kafka client, was used to build a producer that generated synthetic customer data using the Faker library and streamed it to a Kafka topic. A corresponding consumer subscribed to that topic and processed the incoming records in real time. Watching live data flow through the system — seeing timestamps appear, records increment, and the consumer keep pace with the producer — made the abstract concept of event streaming viscerally concrete.

An important architectural insight emerged during this week: Kafka is not a message queue in the traditional sense. Unlike RabbitMQ or ActiveMQ, Kafka retains messages for a configurable retention period, meaning multiple consumers can independently read the same topic at different offsets. This decoupling of producers and consumers — in both time and topology — is what makes Kafka so powerful for building event-driven microservices and data pipelines.

### 4.3 PySpark & Tree-Based Machine Learning Models

Week three combined two related but distinct learning tracks: distributed computing with PySpark, and supervised machine learning with tree-based algorithms. Together, they represented the week with perhaps the greatest volume of practical output — eight separate Jupyter notebooks, each a working model trained and evaluated on real datasets.

Apache Spark's fundamental value proposition is in-memory distributed computation. Where Hadoop MapReduce writes intermediate results to disk between map and reduce phases, Spark caches data in memory across operations, resulting in dramatically faster execution. PySpark, the Python API for Spark, makes this distributed power accessible through a familiar DataFrame interface. The SparkSession object is the entry point, providing methods for reading data, executing SQL queries, and managing distributed transformations.

Working with PySpark's pyspark.ml module for machine learning introduced the Pipeline concept — a sequence of Transformers and Estimators that encapsulates the entire model-building workflow. A Transformer converts one DataFrame into another; an Estimator fits data to produce a Transformer. Chaining these into a single Pipeline ensures that the same preprocessing logic is applied consistently during both training and inference — a subtle but important safeguard against data leakage.

The tree-based machine learning component covered a comprehensive range of algorithms. Decision Trees partition the feature space by recursively finding the split that maximises information gain or minimises Gini impurity. Their key weakness — overfitting on noisy or high-dimensional data — is precisely what ensemble methods address. Random Forests combat overfitting through bagging: building an ensemble of trees, each trained on a random bootstrap sample with a random feature subset at each split. Gradient Boosting takes a sequential approach, with each new tree trained to correct the residual errors of the ensemble so far. XGBoost extends gradient boosting with regularisation, efficient handling of sparse data, and parallelised tree construction.

Across the eight notebooks built during this week, a consistent pattern of experimentation emerged: loading and exploring the dataset, defining the preprocessing pipeline, training with default parameters, evaluating baseline performance, then conducting hyperparameter tuning. This methodical approach to model development, rather than ad hoc experimentation, is a professional practice reinforced by the team's culture.

### 4.4 MLOps Pipeline – MLFlow, DVC & DagsHub

If week three was about building ML models, week four was about managing them — and the distinction matters enormously in production settings. The fourth week was structured around a spike ticket: a time-boxed investigation into a technical concept with the goal of producing a working proof-of-concept. The spike covered the MLOps triad of MLFlow, DVC, and DagsHub.

MLFlow is an open-source platform for managing the complete ML lifecycle. Its four main components address distinct pain points in ML development. MLFlow Tracking provides a systematic logging system for parameters, metrics, and artifacts — eliminating the chaos of scattered notes and inconsistently named model files. MLFlow Projects standardises the packaging of ML code with its runtime environment. MLFlow Models manages model serialisation and serving. MLFlow Model Registry provides a centralised, versioned store for production models with stage transitions (Staging → Production → Archived).

The pipeline built during this week implemented a three-stage ML workflow: a preprocessing stage that read raw CSV data and produced a cleaned version; a training stage that fitted a Random Forest classifier using GridSearchCV and logged all parameters and metrics to MLFlow; and an evaluation stage that loaded the saved model, computed accuracy on a held-out set, and logged the result. The MLFlow tracking UI provided a clean visual interface for comparing runs — a significant improvement over manually parsing log files.

DVC (Data Version Control) addressed a problem that Git alone cannot solve: version controlling large binary files such as datasets and trained models. DVC uses a Git-like command syntax but stores actual file contents in separate remote storage (Google Drive, S3, Azure Blob, etc.), committing only lightweight .dvc metadata files to the Git repository. The ability to checkout an older Git commit and then run dvc checkout to restore the corresponding dataset version is a powerful mechanism for experiment reproducibility.

DagsHub served as the collaborative remote platform, hosting both the Git repository and the DVC remote storage, and providing an integrated MLFlow tracking server. Pushing an experiment from a local Jupyter notebook and immediately seeing it appear in a shared web interface made the value of MLOps tooling tangible in a way that reading about it never could.

## 4.5 Reinforcement Learning & Deep Q-Network

Week five was, by any measure, the most intellectually demanding of the internship. Reinforcement Learning occupies a unique position in the ML landscape — it is neither supervised nor unsupervised. Instead, an agent learns by interacting with an environment, receiving scalar rewards that signal the quality of its actions, and gradually improving its policy to maximise cumulative reward.

The theoretical foundations consumed the first two days. The Markov Decision Process (MDP) formalism — states, actions, transition probabilities, rewards, and discount factors — provided the mathematical scaffolding. Q-learning maintains a table of Q-values Q(s, a) representing the expected cumulative reward of taking action a in state s. The Q-learning update rule iteratively refines these values using the Bellman equation: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max Q(s',a') - Q(s,a)]$. The elegance of this rule — that a Q-value should be consistent with the best Q-value achievable from the next state — is what gives Q-learning its convergence guarantee in discrete, tabular settings.

Deep Q-Networks extend Q-learning to continuous or high-dimensional state spaces where a tabular representation is intractable. A neural network approximates the Q-value function — given a state as input, it outputs a Q-value for each possible action. Two key innovations make DQN stable in practice. Experience Replay stores past transitions in a replay buffer; at each training step, a random mini-batch is sampled — breaking the temporal correlation between consecutive training examples. The epsilon-greedy strategy manages exploration-exploitation: with probability $\varepsilon$ the agent takes a random action, and with probability $1-\varepsilon$ it takes the action with the highest Q-value. $\varepsilon$ decays over time, so the agent explores early and converges to exploitation as it gains confidence.

The CartPole-v1 environment from OpenAI Gym provided the training ground. A pole is balanced on a cart that can move left or right; the state is a four-dimensional vector (cart position, cart velocity, pole angle, pole angular velocity); the action space is binary. An untrained agent fails within seconds; a well-trained DQN agent can balance the pole for the maximum allowed episode length. The implementation process was iterative and humbling. Initial runs produced agents that failed to improve due to poorly chosen hyperparameters. Through systematic experimentation, the right balance was found. Watching the reward curve climb over training episodes — from erratic early performance to consistent long-horizon balance — was one of the most rewarding moments of the entire internship.

### 4.6 NLP with Hugging Face Transformers

The sixth week shifted focus to Natural Language Processing, an area revolutionised by the transformer architecture. The Hugging Face Transformers library, providing a unified interface to hundreds of pre-trained models, was the primary tool for this week's exploration.

The transformer architecture replaced recurrence-based models (LSTMs, GRUs) with a self-attention mechanism that allows every token in a sequence to attend to every other token simultaneously — enabling parallelised training and superior capture of long-range dependencies. Models like BERT were pre-trained on massive text corpora using masked language modelling, producing rich contextual representations that can be fine-tuned for downstream tasks with minimal additional data.

The first part of the week explored four canonical NLP tasks using the Hugging Face pipeline API. Sentiment analysis was performed using DistilBERT fine-tuned on SST-2. Text generation was explored using GPT-2, with creative prompts crafted in the style of Shakespeare to demonstrate contextually coherent text continuation. Question answering used RoBERTa fine-tuned on SQuAD2, extracting answers from provided context passages. Text summarisation employed T5-small fine-tuned on BookSum, condensing lengthy passages into concise summaries.

The second part involved building a real-world sentiment analysis system for Yelp reviews. The pipeline began with web scraping using requests and BeautifulSoup to extract individual review texts from Yelp restaurant pages. These reviews were passed through a multilingual BERT model that scored each review on a scale of 1 to 5. Averaging these scores produced an overall predicted rating, which was compared against the actual Yelp rating. This project demonstrated an important truth about applied NLP: the challenge is rarely the model itself, but the data pipeline around it — scraping is fragile, HTML structures change, and rate limiting is real.

### 4.7 Object Detection with YOLO & Amazon Review Analysis

The final week operated on two parallel tracks: a team-level exploration of YOLO for object detection, and an individual project on Amazon Review Analysis under the mentorship of Krishnav Sir.

YOLO (You Only Look Once) treats object detection as a single regression problem rather than a two-stage pipeline. By dividing the image into a grid and simultaneously predicting bounding boxes and class probabilities for each cell, YOLO achieves remarkable inference speed while maintaining competitive accuracy. The project used the YOLO11n architecture, trained on a dog breed dataset structured in standard YOLO format — with separate image and label directories for train, validation, and test splits, and a dataset.yaml configuration file defining class names and paths.

Training required GPU resources; Google Colab's GPU runtime was used. With 250 training epochs, the model learned meaningful visual features for distinguishing dog breeds — demonstrating the power of transfer learning applied to a domain-specific task. The inference pipeline, loading trained weights and running detection on novel images, produced bounding boxes with class labels that validated the model's learning.

The Amazon Review Analysis individual project served as the capstone of the internship. The task involved exploring multiple approaches to text representation and sentiment classification on a large corpus of Amazon product reviews. Four paradigms were explored: Bag of Words (term-frequency sparse vector), TF-IDF (weighted sparse vector that down-weights common terms), Averaged Word2Vec (dense document representation as the mean of word embeddings), and fine-tuned DistilBERT (contextual deep representation). The comparative analysis revealed a consistent pattern: classical methods with well-tuned classifiers achieved reasonable performance at low computational cost, while DistilBERT delivered the strongest results by leveraging contextual representations that capture nuance and domain-specific language patterns that bag-of-words approaches inherently miss.

Health issues towards the end of the final week caused some delays in completing the fine-tuning experiments. Nevertheless, the foundational work was completed and the learnings from this project consolidated a broad and deep understanding of the NLP landscape — from classical feature engineering to modern transformer-based transfer learning.

# CHAPTER 5: CONCLUSION

Looking back on the seven weeks spent with the PreProd Team, it is clear that this internship was far more than a resume line or a graduation requirement. It was a genuine encounter with the breadth and depth of modern machine learning and data engineering — and with the professional culture that surrounds it.

The technical skills developed across the weeks — Kafka for data streaming, PySpark for distributed computing, tree-based ML for supervised learning, MLFlow and DVC for MLOps, DQN for reinforcement learning, Hugging Face Transformers for NLP, and YOLO for computer vision — represent a portfolio of competencies that span the full stack of contemporary AI engineering. More importantly, they were not learned in isolation but in the context of a working team, with real deliverables, genuine deadlines, and honest feedback.

The Agile framework that structured the internship was perhaps the most durable professional skill acquired. The habit of breaking large ambiguous problems into discrete, estimable tasks; the discipline of daily reflection and communication; the culture of retrospective improvement — these practices transcend any particular technology or domain. They are the operating system on which all other professional skills run.

The challenges encountered — a network configuration that refused to cooperate for three days, a DQN agent that stubbornly failed to converge, a health setback in the final week — were not incidental to the learning but central to it. Persistence, systematic debugging, and the ability to ask for help at the right moment are skills that cannot be taught in a classroom.

The mentorship received from Vansh, Harish Sir, Krishnav Sir, and Mohini transformed what could have been an overwhelming experience into a manageable and enriching one. Their willingness to explain not just the 'how' but the 'why' built the kind of deep understanding that persists long after the specific details fade.

This internship has clarified and strengthened the professional direction ahead. The work done here — particularly in NLP, MLOps, and reinforcement learning — has opened avenues for further exploration and instilled the confidence that complex technical challenges are surmountable with the right tools, the right team, and the right mindset.

## Note on Internship Schedule

The seven-week internship period was not entirely contiguous. In keeping with institutional academic commitments, a period of scheduled activity was designated by VIT Bhopal University (approximately mid-September through October). Additionally, one week was lost towards the end of the internship duration due to personal illness. These periods are not reflected in the weekly activity tables of this report, which document active working weeks only.

The structured learning and deliverables described in this report represent seven complete, productive working weeks within the broader internship duration.

## Bibliography


1. Apache Software Foundation. (2024). Apache Kafka Documentation. https://kafka.apache.org/documentation/

2. Apache Software Foundation. (2024). Apache Spark Documentation. https://spark.apache.org/docs/latest/

3. MLflow. (2024). MLflow Documentation. https://mlflow.org/docs/latest/index.html

4. DVC. (2024). Data Version Control Documentation. https://dvc.org/doc

5. DagsHub. (2024). DagsHub Documentation. https://dagshub.com/docs/

6. Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518, 529–533.

7. Hugging Face. (2024). Transformers Documentation. https://huggingface.co/docs/transformers/index

8. Vaswani, A., et al. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems, 30.

9. Ultralytics. (2024). YOLO Documentation. https://docs.ultralytics.com/

10. Devlin, J., et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL-HLT 2019.

11. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. ACM SIGKDD 2016.

12. Breiman, L. (2001). Random Forests. Machine Learning, 45, 5–32.

13. OpenAI. (2024). Gym Documentation. https://www.gymlibrary.dev/

14. Quixstreams. (2024). Quixstreams Documentation. https://quix.io/docs/quix-streams/

15. Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.